

JavaScript for Grade 10 (FBISE)

Unit 3: Programming Fundamentals – Lecture Notes & Study Guide

Lecture 1: Introduction to JavaScript & Web Development

1.1 What is JavaScript?

JavaScript is a **programming language** that runs in the web browser and makes web pages **interactive and dynamic**. It is a **client-side scripting language** used to create dynamic websites.

Key Characteristics

Behavior: Adds interactive and dynamic capabilities to web pages.

Client-Side Execution: Runs directly inside the user's web browser.

Dynamic Content: Updates page content without requiring a full page reload.

Event Handling: Processes user interactions such as button clicks, scrolling, and form entries.

The Three Pillars of Web Development

Technology	Role	Examples
HTML	Structure	Headings, paragraphs, tables, forms
CSS	Style	Colors, fonts, layout, animations
JavaScript	Behavior	Interactivity, dynamic content, calculations

Front-End vs. Back-End

Front-End (Client-Side)	Back-End (Server-Side)
HTML, CSS, JavaScript	Python, PHP, Ruby, Java, Node.js
Runs inside the browser	Runs on the web server
User interface & visual elements	Data storage, authentication & business logic

1.2 Why JavaScript is Important

Conventional static websites display identical content to all users. JavaScript introduces dynamic capabilities that make web applications responsive and interactive.

Feature	Explanation	Example
Interactivity	Responds to user actions	Button changes color on hover/click
Data Handling	Processes and filters information	Sorting product items in a list
Dynamic Updates	Refreshes content without reloading	Live notifications or feeds updating automatically
Complex UI	Powers rich graphical interfaces	Drop-down menus, image carousels, modal popups

1.3 Where to Write JavaScript

JavaScript can be included in an HTML document using three methods:

1. Inline JavaScript

HTML

```
<button onclick="alert('Hello!')">Click Me</button>
```

2. Internal JavaScript

HTML

```
<script>
  alert("Hello World!");
</script>
```

3. External JavaScript

HTML

```
<script src="script.js"></script>
```

Summary / Board Work

None

JAVASCRIPT = Behavior of Web Page

HTML -> Structure (skeleton)

CSS -> Style (presentation)

JS -> Behavior (actions)

Front-End: HTML, CSS, JS (browser)

Back-End: Python, PHP, Java (server)

Methods of Inclusion:

1. Inline -> onclick="..."

2. Internal -> <script>...</script>

3. External -> <script src="file.js"></script>

Lecture 2: JavaScript Basics – Variables, Data Types & Control Flow

2.1 Variables in JavaScript

A **variable** is a named container used for storing data values in memory.

Variable Declaration Keywords

Keyword	Scope / Description	Example
var	Legacy standard, function-scoped	var x = 10;
let	Modern standard, block-scoped	let y = 20;
const	Block-scoped, value cannot be reassigned	const z = 30;

Example Code

JavaScript

```
let studentName = "Ali";
```

```
const age = 15;
```

```
var marks = 85;
```

2.2 Data Types in JavaScript

Data Type	Description	Example
String	Textual sequence	"Hello", "World"
Number	Integers and floating-point decimals	10, 3.14, -5
Boolean	Logical truth value	true, false
Array	Ordered collection of values	[1, 2, 3]
Object	Key-value property collection	{name: "Ali", age: 15}
Null	Intentional empty value	null
Undefined	Uninitialized variable	undefined

2.3 Output Methods

Method	Description	Example
console.log()	Outputs message to browser console	console.log("Hello");
alert()	Displays modal alert dialog	alert("Hello");
document.write()	Writes directly to HTML document	document.write("Hello");
innerHTML	Modifies content of an HTML element	document.getElementById("demo").innerHTML = "Hi";

2.4 Operators

Arithmetic Operators

JavaScript

```

let a = 10, b = 3;
a + b;    // 13 (Addition)
a - b;    // 7  (Subtraction)
a * b;    // 30 (Multiplication)
a / b;    // 3.33 (Division)
a % b;    // 1  (Modulus / Remainder)
a++;      // 11 (Increment)
a--;      // 9  (Decrement)

```

Comparison Operators

JavaScript

```
5 == "5";    // true  (Loose equality - converts types)
5 === "5";   // false (Strict equality - checks type and value)
5 != 3;      // true  (Not equal)
5 > 3;       // true  (Greater than)
5 < 3;       // false (Less than)
5 >= 5;      // true  (Greater than or equal to)
5 <= 3;      // false (Less than or equal to)
```

Logical Operators

JavaScript

```
true && false; // false (Logical AND)
true || false; // true  (Logical OR)
!true;         // false (Logical NOT)
```

2.5 Conditionals

if-else Statement

JavaScript

```
let marks = 75;
if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

else-if Ladder

JavaScript

```
let grade;
if (marks >= 80) {
  grade = "A";
} else if (marks >= 60) {
  grade = "B";
} else if (marks >= 40) {
  grade = "C";
} else {
  grade = "F";
}
```

switch Statement

JavaScript

```
let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Other day");
}
```

2.6 Loops

for Loop

JavaScript

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}  
// Output: 1 2 3 4 5
```

while Loop

JavaScript

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

do-while Loop

JavaScript

```
let i = 1;  
do {  
  console.log(i);  
  i++;  
} while (i <= 5);
```

for-of Loop (Arrays)

JavaScript

```
let fruits = ["apple", "banana", "orange"];  
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

Summary / Board Work

None

VARIABLES:

```
var x = 10;    (legacy, function-scoped)
let y = 20;    (reassignable, block-scoped)
const z = 30;  (constant, block-scoped)
```

DATA TYPES:

String, Number, Boolean, Array, Object, Null, Undefined

OPERATORS:

Arithmetic: + - * / % ++ --

Comparison: == === != > < >= <=

Logical: && || !

CONDITIONALS:

```
if (cond) { ... } else if { ... } else { ... }
switch (val) { case x: ... break; default: ... }
```

LOOPS:

```
for (let i=0; i<5; i++) { ... }
while (condition) { ... }
do { ... } while (condition);
```

Lecture 3: JavaScript Arrays

3.1 What is an Array?

An **array** is a data structure that stores an ordered collection of elements under a single variable name. Each element is accessed via a zero-based **index**.

Creating Arrays

JavaScript

```
const fruits = ["apple", "banana", "orange"];
const numbers = [10, 20, 30, 40];
```

```
const mixed = ["Ali", 15, true, 3.14];
```

Array Indexing

JavaScript

```
fruits[0]; // "apple"  
fruits[1]; // "banana"  
fruits[2]; // "orange"
```

3.2 Array Properties and Methods

Method / Property	Description	Example
length	Returns number of elements	fruits.length -> 3
push()	Appends item to the end	fruits.push("mango")
pop()	Removes last item	fruits.pop()
shift()	Removes first item	fruits.shift()
unshift()	Prepends item to the start	fruits.unshift("grape")
indexOf()	Finds index of element	fruits.indexOf("banana") -> 1
includes()	Checks if element exists	fruits.includes("apple") -> true
concat()	Merges two arrays	arr1.concat(arr2)
slice()	Extracts portion of array	fruits.slice(1, 3)
splice()	Adds/removes elements at index	fruits.splice(1, 1)
sort()	Sorts array elements	fruits.sort()
reverse()	Reverses array order	fruits.reverse()

3.3 Looping Through Arrays

Standard for Loop

JavaScript

```
let fruits = ["apple", "banana", "orange"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

for-of Loop

JavaScript

```
for (let fruit of fruits) {
  console.log(fruit);
}
```

forEach Method

JavaScript

```
fruits.forEach(function(fruit) {
  console.log(fruit);
});
```

3.4 Practical Example: Sum of First 10 Integers

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Sum of First 10 Integers</title>
</head>
<body>
  <button onclick="calculateSum()">Calculate Sum</button>
  <p id="numberDisplay"></p>
  <p id="sumDisplay"></p>
```

```

<script>
  function calculateSum() {
    let sum = 0;
    let numbers = [];

    for (let i = 1; i <= 10; i++) {
      sum += i;
      numbers.push(i);
    }

    document.getElementById("sumDisplay").textContent =
      "The sum of the first 10 integers is: " + sum;
    document.getElementById("numberDisplay").textContent =
      "The first 10 integers are: [" + numbers.join(", ") + "];"
  }
</script>
</body>
</html>

```

Output

None

```

The first 10 integers are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
The sum of the first 10 integers is: 55

```

3.5 Practical Example: Dynamic Shopping List Table

```

HTML
<!DOCTYPE html>
<html>
<head>
  <title>Shopping List</title>
  <style>
    table { border-collapse: collapse; width: 100%; }

```

```

        th, td { padding: 10px; border: 1px solid lightgray;
text-align: left; }
    </style>
</head>
<body>
    <h1>My Shopping List</h1>
    <input type="text" id="newItem" placeholder="Enter item name">
    <button onclick="addItem()">Add Item</button>
    <br><br>
    <table id="shoppingListTable"></table>

    <script>
        let shoppingList = [];

        function addItem() {
            const input = document.getElementById("newItem");
            const newItemName = input.value.trim();
            if (newItemName !== "") {
                shoppingList.push(newItemName);
                input.value = "";
                updateShoppingListTable();
            } else {
                alert("Please enter an item name!");
            }
        }

        function updateShoppingListTable() {
            const table = document.getElementById("shoppingListTable");
            table.innerHTML = "<tr><th>Shopping List Items</th></tr>";
            for (let item of shoppingList) {
                table.innerHTML += `<tr><td>${item}</td></tr>`;
            }
        }
    </script>
</body>
</html>

```

Summary / Board Work

None

ARRAY = Collection of items (0-indexed)

Declaration: `const fruits = ["apple", "banana", "orange"];`

Key Methods:

`push()` -> Add to end
`pop()` -> Remove from end
`shift()` -> Remove from start
`unshift()` -> Add to start
`indexOf()` -> Find index position
`length` -> Total count of items

Looping Methods:

`for (let i = 0; i < arr.length; i++) { ... }`
`for (let item of arr) { ... }`
`arr.forEach(item => { ... });`

Lecture 4: Functions in JavaScript

4.1 What is a Function?

A **function** is a reusable block of code designed to perform a specific task. Functions execute when they are invoked (called).

Benefits of Functions

Reusability: Write code once and execute it multiple times.

Organization: Decompose complex problems into manageable sub-tasks.

Maintainability: Improves code clarity and simplifies debugging.

Testability: Allows isolated testing of specific functionality.

4.2 Defining Functions

Basic Function

JavaScript

```
function greet() {  
  console.log("Hello, World!");  
}  
greet(); // Output: Hello, World!
```

Function with Parameters

JavaScript

```
function greetUser(name) {  
  console.log("Hello, " + name + "!");  
}  
greetUser("Ali"); // Output: Hello, Ali!  
greetUser("Sara"); // Output: Hello, Sara!
```

Function with Return Value

JavaScript

```
function add(a, b) {  
  return a + b;  
}  
let result = add(5, 3);  
console.log(result); // Output: 8
```

Arrow Functions (ES6)

JavaScript

```
const add = (a, b) => a + b;  
console.log(add(5, 3)); // Output: 8
```

4.3 Practical Function Examples

Example 1: Calculate Area

JavaScript

```
function calculateArea(length, width) {  
  return length * width;  
}  
let area = calculateArea(10, 5);  
console.log("Area: " + area); // Output: Area: 50
```

Example 2: Item Existence Check

JavaScript

```
const list = ["apples", "bread", "milk", "cheese"];  
  
function findItem(itemName) {  
  return list.includes(itemName);  
}  
  
console.log(findItem("milk")); // true  
console.log(findItem("eggs")); // false
```

Example 3: Returning Multiple Values via Object

JavaScript

```
function calculateSum() {  
  let sum = 0;  
  let numbers = [];  
  for (let i = 1; i <= 10; i++) {  
    sum += i;  
    numbers.push(i);  
  }  
  return { sum: sum, numbers: numbers };  
}
```

```
}
```

```
let result = calculateSum();  
console.log(result.sum);      // 55  
console.log(result.numbers); // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Summary / Board Work

None

FUNCTION = Reusable code block

Syntax:

```
function functionName(param1, param2) {  
    // execute logic  
    return value;  
}
```

Arrow Function (ES6):

```
const add = (a, b) => a + b;
```

Key Advantages:

1. Reusability 2. Organization 3. Readability 4. Simplified Testing

Lecture 5: DOM Manipulation (Document Object Model)

5.1 What is the DOM?

The **Document Object Model (DOM)** is an API for HTML documents. It presents the page structure as a hierarchical tree of objects that JavaScript can dynamically inspect and modify.

DOM Tree Structure Structure

None

```
document
├── html
│   ├── head
│   │   └── title
│   └── body
│       ├── h1
│       ├── p
│       └── button
```

5.2 Selecting DOM Elements

Method	Selection Target	Example
getElementById()	Single element by unique ID	document.getElementById("demo")
getElementsByClassName()	Collection by class name	document.getElementsByClassName("box")
getElementsByTagName()	Collection by HTML tag	document.getElementsByTagName("p")
querySelector()	First matching CSS selector	document.querySelector(".box")
querySelectorAll()	NodeList of all CSS matches	document.querySelectorAll("p")

5.3 Modifying Elements & Styles

Modifying Content

JavaScript

```
// HTML interpretation
document.getElementById("demo").innerHTML = "<b>Hello</b>";

// Plain text only
document.getElementById("demo").textContent = "Hello";
```

Modifying Styles & Classes

JavaScript

```
// Inline styles
document.getElementById("demo").style.color = "red";
document.getElementById("demo").style.fontSize = "20px";

// Class manipulation
document.getElementById("demo").classList.add("active");
document.getElementById("demo").classList.remove("active");
```

5.4 Creating and Appending Elements

JavaScript

```
// Create element
let newParagraph = document.createElement("p");
newParagraph.textContent = "This is a dynamically added
paragraph.";

// Append to container element
document.getElementById("container").appendChild(newParagraph);
```

5.5 Practical Example: Dynamic List Generation

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Shopping List</title>
</head>
<body>
  <h1>My Shopping List</h1>
  <ul id="shoppingList"></ul>
```

```

<script>
  const shoppingItems = ["Milk", "Bread", "Eggs", "Apples"];

  function updateList() {
    const listElement = document.getElementById("shoppingList");
    listElement.innerHTML = "";
    for (let item of shoppingItems) {
      const listItem = document.createElement("li");
      listItem.textContent = item;
      listElement.appendChild(listItem);
    }
  }
  updateList();
</script>
</body>
</html>

```

Summary / Board Work

None

DOM = Document Object Model (HTML Element Tree)

Selection Methods:

```

document.getElementById("id")
document.querySelector(".class")
document.querySelectorAll("tag")

```

Modification:

```

element.innerHTML = "<b>text</b>";
element.textContent = "text";
element.style.color = "blue";
element.classList.add("active");

```

Element Creation:

```

let li = document.createElement("li");

```

```
li.textContent = "New Item";  
parentElement.appendChild(li);
```

Lecture 6: Event Handling

6.1 What is an Event?

An **event** is a signal triggered by user actions or browser state changes, such as:

- Clicking elements or buttons
- Pressing keyboard keys
- Submitting HTML forms
- Document or asset loading
- Mouse movements and hovering

6.2 Event Handling Methods

Method 1: Inline HTML Attribute

HTML

```
<button onclick="alert('Clicked!')">Click Me</button>
```

Method 2: addEventListener (Recommended Standard)

JavaScript

```
document.getElementById("myButton").addEventListener("click",  
function() {  
    alert("Clicked!");  
});
```

Method 3: DOM Property Assignment

JavaScript

```
document.getElementById("myButton").onclick = function() {  
    alert("Clicked!");  
};
```

```
};
```

6.3 Common Events Table

Event Name	Description
click	Fires when element is clicked
dblclick	Fires on double-click
mouseover / mouseout	Fires when mouse enters or leaves element
keydown / keyup	Fires on key press or release
submit	Fires when form is submitted
change	Fires when input field value changes

6.4 Practical Example: Click Counter

```
HTML
<!DOCTYPE html>
<html>
<head>
  <title>Click Counter</title>
</head>
<body>
  <h1>Click Counter</h1>
  <button id="clickBtn">Click Me</button>
  <p id="countDisplay">0</p>

  <script>
    let count = 0;
    document.getElementById("clickBtn").addEventListener("click",
function() {
    count++;
    document.getElementById("countDisplay").textContent = count;
  });
  </script>
```

```
</body>
</html>
```

6.5 Practical Example: Form Validation

```
HTML
<!DOCTYPE html>
<html>
<head>
  <title>Form Validation</title>
</head>
<body>
  <form id="myForm">
    <label for="name">Name:</label>
    <input type="text" id="name" required><br><br>
    <label for="email">Email:</label>
    <input type="email" id="email" required><br><br>
    <button type="submit">Submit</button>
  </form>

  <script>
    document.getElementById("myForm").addEventListener("submit",
function(event) {
  let name = document.getElementById("name").value.trim();
  let email = document.getElementById("email").value.trim();
  if (name === "" || email === "") {
    event.preventDefault();
    alert("Please fill out all required fields!");
  } else {
    alert("Form submitted successfully!");
  }
});
  </script>
</body>
</html>
```

Summary / Board Work

None

EVENT = User interaction or browser signal

Methods to Attach Handlers:

1. HTML: `<button onclick="myFunc()">`
2. JS Standard: `element.addEventListener("click", myFunc);`
3. JS Property: `element.onclick = myFunc;`

Common Event Types:

`click, dblclick, mouseover, mouseout, keydown, keyup, submit, change`

Lecture 7: Basic Algorithms – Linear Search

7.1 Linear Search Algorithm

Linear Search sequentially inspects each element in an array from start to finish until the target value is located or the array ends.

Algorithm Steps

- Start at the first element (index 0).
- Compare current element with target value.
- If match is found, return current index.
- If no match, advance to next element.
- Repeat until target is found or array end is reached.
- If target is absent, return -1.

JavaScript Implementation

JavaScript

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i; // Found at index i  
    }  
  }  
}
```

```

    }
  }
  return -1; // Not found
}

let numbers = [10, 20, 30, 40, 50];
console.log(linearSearch(numbers, 30)); // 2
console.log(linearSearch(numbers, 60)); // -1

```

7.2 Practical Example: Searching Items in Shopping List

JavaScript

```

const shoppingList = ["apples", "bread", "milk", "cheese"];

function searchItem(itemName) {
  for (let item of shoppingList) {
    if (item === itemName) {
      return `${itemName} is in your shopping list!`;
    }
  }
  return `${itemName} is not in your shopping list.`;
}

console.log(searchItem("milk"));

```

Output

None

```
milk is in your shopping list!
```

7.3 Practical Example: Interactive Course Search UI

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Course Search</title>
</head>
<body>
  <h1>Course Finder</h1>
  <p id="courseList"></p>
  <input type="text" id="searchInput" placeholder="Enter course
name">
  <button onclick="searchCourse()">Search</button>
  <p id="searchResult"></p>

  <script>
    const courses = [
      "Computer Science", "Mathematics", "Physics", "Chemistry",
      "Biology", "English", "Urdu", "Islamiat", "Pakistan Studies",
"Economics"
    ];

    document.getElementById("courseList").textContent =
      "Available Courses: " + courses.join(", ");

    function searchCourse() {
      const query =
document.getElementById("searchInput").value.trim().toLowerCase();
      const match = courses.find(c => c.toLowerCase() === query);
      document.getElementById("searchResult").textContent = match
        ? "Course found: " + match
        : "Course not found!";
    }
  </script>
</body>
</html>
```

Summary / Board Work

None

LINEAR SEARCH:

- Inspect each element sequentially
- Return index if target matches
- Return -1 if target is absent

Algorithm Logic:

```
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] === target) return i;  
}  
return -1;
```

Lecture 8: Testing and Debugging

8.1 Unit Testing

Unit testing involves isolating individual modules or functions and testing them independently to verify expected behavior.

Importance of Unit Testing

- Identifies bugs early in the development cycle.
- Ensures software reliability and code quality.
- Allows refactoring without breaking existing logic.
- Serves as functional documentation for codebase logic.

Unit Test Example

JavaScript

```
const list = ["apples", "bread", "milk", "cheese"];  
  
function findItem(itemName) {  
  return list.includes(itemName);  
}
```

```
// Unit Test 1: Existing item
console.log(findItem("milk") === true ? "Test 1 Passed" : "Test 1
Failed");

// Unit Test 2: Non-existing item
console.log(findItem("eggs") === false ? "Test 2 Passed" : "Test 2
Failed");
```

Output

```
None
Test 1 Passed
Test 2 Passed
```

8.2 Debugging & Error Categories

Debugging is the systematic process of identifying, analyzing, and removing software errors (bugs).

Error Type	Description	Example
Syntax Error	Violates programming language grammar	Missing closing bracket or quote
Logical Error	Code executes but produces incorrect output	Using + operator instead of -
Runtime Error	Error occurs during code execution	Calling undefined function or null access

8.3 Debugging Techniques

Console Logging: Use `console.log()` to print intermediate state variables during execution.

Breakpoints: Pause execution on designated lines inside DevTools to inspect environment state.

Watchpoints: Monitor designated variable values for dynamic changes.

Step Execution: Execute instructions line-by-line (Step-Over/Step-Into) to trace control flow.

8.4 Example: Debugging Logic Errors

Buggy Code vs. Fixed Code

JavaScript

```
// BUGGY: Loop initialized at i = 10 instead of i = 1
for (let i = 10; i <= 10; i++) {
  sum += i;
}
```

```
// FIXED: Correct loop initialization
for (let i = 1; i <= 10; i++) {
  sum += i;
}
```

Summary / Board Work

None

UNIT TESTING:

- Verify individual components in isolation
- Check expected vs actual outputs

DEBUGGING:

- Process of locating & fixing bugs

Error Types:

1. Syntax Error -> Grammar violations
2. Logical Error -> Incorrect output/behavior
3. Runtime Error -> Execution crash

Debugging Tools & Methods:

- console.log()
- Breakpoints & Watchpoints
- Line-by-line step execution

Lecture 9: Complete Practical Applications

9.1 Hello World Web Project

HTML (index.html)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Change Message</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div id="message">Hello World!</div>
  <button id="change-message-button">Change Message</button>
  <script src="script.js"></script>
</body>
</html>
```

CSS (style.css)

CSS

```
body {
  font-family: Arial, sans-serif;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
  background-color: #f0f0f0;
```

```

}

#message {
  color: #333;
  font-size: 2em;
  margin-bottom: 20px;
}

button {
  padding: 10px 20px;
  font-size: 1em;
  cursor: pointer;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 5px;
}

button:hover {
  background-color: #45a049;
}

```

JavaScript (script.js)

```

JavaScript
document.addEventListener("DOMContentLoaded", function() {
  const messageDiv = document.getElementById("message");
  const button = document.getElementById("change-message-button");

  button.addEventListener("click", function() {
    if (messageDiv.textContent === "Hello World!") {
      messageDiv.textContent = "I Love my country, Pakistan";
    } else {
      messageDiv.textContent = "Hello World!";
    }
  })
}

```

```
});  
});
```

9.2 Shopping List Application

```
HTML  
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Shopping List</title>  
  </head>  
  <body>  
    <h1>My Shopping List</h1>  
    <ul id="shoppingList"></ul>  
    <input type="text" id="newItemInput" placeholder="Enter new  
item">  
    <button onclick="addItem()">Add Item</button>  
  
    <script>  
      const shoppingItems = ["Milk", "Bread", "Eggs", "Apples"];  
  
      function addItem() {  
        const input = document.getElementById("newItemInput");  
        const newItem = input.value.trim();  
        if (newItem !== "") {  
          shoppingItems.push(newItem);  
          input.value = "";  
          updateList();  
        }  
      }  
  
      function updateList() {  
        const listElement = document.getElementById("shoppingList");  
        listElement.innerHTML = "";  
        for (let item of shoppingItems) {
```

```

        const listItem = document.createElement("li");
        listItem.textContent = item;
        listElement.appendChild(listItem);
    }
}
updateList();
</script>
</body>
</html>

```

Unit Summary & Key Concepts

Topic	Key Concepts
Introduction	Front-end vs back-end, role of JS in web development
Variables	var, let, const keywords & scoping rules
Data Types	String, Number, Boolean, Array, Object, Null, Undefined
Operators	Arithmetic, comparison, and logical operators
Conditionals	if-else, else-if ladder, switch constructs
Loops	for, while, do-while, and for-of loops
Arrays	Creation, indexing, push, pop, shift, unshift, length, slice, concat
Functions	Declarations, parameters, return values, arrow functions
DOM Manipulation	getElementById, textContent, innerHTML, createElement, appendChild
Events	addEventListener, onclick, common event types
Algorithms	Linear search algorithm implementation
Testing	Unit testing principles and assertions
Debugging	console.log, breakpoints, watchpoints, error categories

Exam Reference – Essential Code Snippets

1. Array Declaration and Iteration

JavaScript

```
const fruits = ["apple", "banana", "orange"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

2. Function Declaration and Invocation

JavaScript

```
function add(a, b) {
  return a + b;
}
let result = add(5, 3);
```

3. DOM Element Selection and Content Update

JavaScript

```
document.getElementById("demo").innerHTML = "<b>Hello</b>";
document.getElementById("demo").textContent = "Hello";
```

4. Event Listener Registration

JavaScript

```
document.getElementById("btn").addEventListener("click", function()
{
  alert("Clicked!");
});
```

5. Essential Array Methods

JavaScript

```
arr.push(item);      // Append to end
arr.pop();           // Remove from end
arr.length;          // Element count
arr.indexOf(item);   // Find index
arr.includes(item);  // Check existence
```

6. Simple Unit Assertion

JavaScript

```
if (findItem("milk") === true) {
  console.log("Test Passed");
} else {
  console.log("Test Failed");
}
```

7. Linear Search Implementation

JavaScript

```
for (let i = 0; i < arr.length; i++) {
  if (arr[i] === target) return i;
}
return -1;
```